

Name: \_\_\_\_\_

Class: \_\_\_\_\_

Mr. Rawson • GCSE Computer Science



Year 11 2026

## Programming 5 — String Manipulation

*A String is not one thing. It is a row of characters with numbered positions — and once you see it that way, you can take any piece of it you like.*

### Every character has a position, and counting starts at 0

We will use one string for the whole sheet. It is the one OCR uses in the specification, and it has **no space** in it.

```
subject = "ComputerScience"
```

|   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |
|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|
| C | o | m | p | u | t | e | r | S | c | i  | e  | n  | c  | e  |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |

**Fifteen characters, numbered 0 to 14.** The last position is always one less than the length, and that single fact causes more bugs than anything else on this sheet.

**Now try** • `print(subject[0])` — which character comes out?

**Now try** • `print(subject[14])` — and that one?

**Now try** • Now try `print(subject[15])`. *What happens, and why?*

### Length

| ERL                         | Gives | Python                    |
|-----------------------------|-------|---------------------------|
| <code>subject.length</code> | 15    | <code>len(subject)</code> |

**In ERL there are no brackets after `.length`.** In Python it is a function you wrap around the string instead. The two languages disagree about this and the exam prints the ERL version.

### Taking a piece out — slicing

Three ways, and ERL names them plainly.

| ERL                                 | Gives   | Python                    |
|-------------------------------------|---------|---------------------------|
| <code>subject.left(4)</code>        | "Comp"  | <code>subject[:4]</code>  |
| <code>subject.right(3)</code>       | "nce"   | <code>subject[-3:]</code> |
| <code>subject.substring(3,5)</code> | "puter" | <code>subject[3:8]</code> |

substring is the one to get wrong. Its two numbers are **start** and **how many** — *not* start and end.

`subject.substring(3,5)` means *begin at position 3, take 5 characters*: positions 3, 4, 5, 6, 7 — which is "puter".

Python's slice is start and *end*, so the same job is `subject[3:8]`. Read an ERL substring as a Python slice and you will take the wrong number of characters every time.

**Q1.** Using `subject = "ComputerScience"`, give the output. [5]

| Code                                       | Output |
|--------------------------------------------|--------|
| <code>print(subject.left(8))</code>        |        |
| <code>print(subject.right(7))</code>       |        |
| <code>print(subject.substring(0,3))</code> |        |
| <code>print(subject.length)</code>         |        |
| <code>print(subject.substring(8,7))</code> |        |

**Now try** • Go to [101computing.net/string-slicing-in-python](http://101computing.net/string-slicing-in-python).

**Now try** • **Use the slider.** Drag the two handles and watch the slice change. Find the settings that give you "puter".

**Now try** • Then find the settings for "Science" *without* counting to 8 by hand.

## Joining — concatenation

ERL has exactly one way to join text: `+`. No others.

```
print("Hello" + "Everyone")      # HelloEveryone    -- 13 characters, no space
print("Hello " + name)           # the space is yours to put in
```

**+ will not join text to a number.** `"Year " + 11` is a `TypeError`. You must cast first — `"Year " + str(11)` — which is exactly the job `str()` was invented for.

**Q2.** One line below fails. Which, and how would you fix it? [2]

```
age = 15
print("You are " + age + " years old")
```

## Changing case, and character codes

| ERL                        | Gives             | Python                       |
|----------------------------|-------------------|------------------------------|
| <code>subject.upper</code> | "COMPUTERSCIENCE" | <code>subject.upper()</code> |
| <code>subject.lower</code> | "computerscience" | <code>subject.lower()</code> |
| <code>ASC('A')</code>      | 65                | <code>ord('A')</code>        |
| <code>CHR(97)</code>       | 'a'               | <code>chr(97)</code>         |

**Every character is stored as a number.** 'A' is 65, 'B' is 66, and 'a' is 97. So there are exactly **32** between a capital and its lowercase twin — which is why `CHR(ASC('a') - 32)` gives you 'A'.

**The two languages use different words for the same pair.** `ord` is short for *ordinal*, meaning “position number”, and `chr` is short for *character*. They do exactly what `ASC` and `CHR` do.

**There is only ONE table, and `ASC` and `CHR` read it in opposite directions.** Give it a character, get a number; give it a number, get a character. So `CHR(ASC("q"))` hands you back "q".

**This table is called ASCII.** You do not have to learn it. What you do have to know is that it is **in order** — and these are the only landmarks worth remembering:

| Codes   | Characters |                               |
|---------|------------|-------------------------------|
| 32      | space      |                               |
| 48--57  | 0 to 9     | so '7' is $48 + 7 = 55$       |
| 65--90  | A to Z     | 26 in a row, no gaps          |
| 97--122 | a to z     | exactly 32 above the capitals |

**Everything below 32 is a *control character*** — an instruction rather than a symbol. They come from teleprinters and they are still here. `CHR(9)` is a **tab**, `CHR(10)` is a **new line**, `CHR(7)` once rang a physical bell. *A control character prints as nothing visible, which is why one in the middle of your output is so confusing.*

**In the exam you get NONE of this.**

OCR gives you **no ASCII table, no reference card and no data sheet** — and **no calculator**, in either paper. You walk in with a pen.

**That is deliberate, and it tells you what is actually examined.** The specification says plainly that memorising character codes is *not required*. If a question needs a specific code, **the question will give it to you.**

**What you must know without being told is that the table is in order** — the code for B is one more than the code for A. That single fact is the examinable one, and it is the whole reason a Caesar shift works by adding.

*So use the table above in class freely. Just do not plan on it being there in June.*

**Now try** • `print(ord("A"))` and `print(ord("a"))`. What is the gap?

**Now try** • `print(chr(66))`, then `print(chr(98))`.

**Now try** • Predict `print(ord("Z") - ord("A"))` before you run it.

## Putting it together — the school username

This is where all of it earns its keep. Schools generate usernames from a rule, not by hand.

**The rule:** two-digit year group, then the first letter of the first name, then the surname — all lowercase.

So **Alex Rawson** in **Year 7** becomes 07arawson.

**Q3.** Name the string operation each step needs. [4]

| Step                                     | Which operation? |
|------------------------------------------|------------------|
| Turn the year group 7 into the text "07" |                  |
| Take the A from "Alex"                   |                  |
| Make everything lowercase                |                  |
| Stick the three parts together           |                  |

*Row 1 is the awkward one. Year 7 gives "7", which is one character, not two — so a single-digit year needs a "0" putting in front of it. Year 11 does not. That is a decision, which means selection.*

### Challenge — What's My Username? [101computing.net/whats-my-username](https://101computing.net/whats-my-username)

Write the program. The page gives you a **table of test data** — do not call it finished until every row comes out right.

**Work up to it in stages**, and run it after each one:

1. Ask for the first name, the surname and the year group, and print them back.
2. Print just the first letter of the first name.
3. Join the three pieces together in the right order.
4. Make the whole thing lowercase.
5. **Then** deal with the single-digit year groups.

**Leave step 5 until last.** It is the only genuinely awkward part, and four working steps behind you is a much better place to meet it from.

**If you finish:** the String Slicing page has its own challenge — validating a username rather than building one. It checks the length, looks for an underscore, and works out whether the user is a student, a teacher or admin staff.

*Total: 11 marks on the written questions. **Keep the reference sheet beside you** — Pseudocode, ERL and Python, on the class page. Its string table uses this same example, and it is where the ERL-to-Python differences are set out in full.*